

APEX CODE CHANGE REVIEW

Google Drive Photo Authentication Fix | Story 00010716

Environment	Probo Medical UAT
Scope	Three Apex classes
Deployment	0AfjH0000000T6rSAE
Verification	3/3 components; 4/4 tests passed
Prepared for	Technical and business review
Status	Current deployed UAT code

Code-only comparison. No Google Drive folder setup is included.

1. What was wrong

Google authentication temporarily sends the user away from Salesforce. The old code did not reliably carry the Evaluation ID through that trip. When the user came back, the controller could fall back to the most recently modified Evaluation, which meant the wrong record could be selected.

The controller also treated a full Google Drive URL as if it were already a clean folder ID. Missing records, short filenames, or incomplete Drive responses could cause query, substring, or null-reference failures.

2. How we fixed it

Simple explanation: We gave the Google round trip a claim ticket. The claim ticket is the exact Salesforce Evaluation ID. When Google sends the user back, Salesforce reads that ticket, validates it, and resumes the correct Evaluation instead of guessing.

1. Carry the exact Evaluation ID inside the OAuth state parameter.
2. Restore and validate that ID after Google returns to Salesforce.
3. Remove the fallback that selected the user's most recently modified Evaluation.
4. Convert a complete Drive folder URL into a clean folder ID.
5. Stop safely with readable messages when required records or folder data are missing.
6. Strengthen the test class with a target Evaluation and a control Evaluation.

3. Apex scope

Exactly three Apex class files changed. Their three .cls-meta.xml files did not change, and no Visualforce page was changed in this deployment.

APEX CLASS	OLD UAT	CURRENT UAT	PURPOSE
<code>cAuthURIForEval</code>	-1 lines	+11 lines	Carry the Evaluation ID through Google OAuth.
<code>cGoogleAppAuthenticationWithSalesforce</code>	-134 lines	+153 lines	Restore the correct record, clean the folder ID, and fail safely.
<code>GoogleAuthTestClass</code>	-7 lines	+92 lines	Prove the callback updates only the intended Evaluation.

4. Important boundary

These changes fix callback record selection, folder-ID normalization, defensive error handling, and automated coverage. They do not create a new Cap field and do not change the pre-existing image-to-field assignment logic. This document intentionally covers code changes only.

5. Side-by-side Apex diff

Red-tinted lines are old UAT code that was removed or replaced. Green-tinted lines are the current UAT code that was added or replaced. Uncolored lines provide nearby context.

5.1 cAuthURIForEval

Verified change size: +11 lines / -1 lines.

Change 1: Accept the Evaluation ID

OLD UAT CODE	NEW UAT CODE
<pre> 5 public String AuthenticationURI=''; 6 public cAuthURIForEval(String Clientkey,String redirect_uri) </pre>	<pre> 5 public String AuthenticationURI=''; 6 public cAuthURIForEval(String Clientkey,String redirect_uri) 7 + { 8 + this(Clientkey, redirect_uri, null); 9 + } 10 + 11 + public cAuthURIForEval(String Clientkey, String redirect_uri, String evaluationId) 12 { 13 String key = EncodingUtil.urlEncode(Clientkey,'UTF-8'); </pre>
<pre> 7 { 8 String key = EncodingUtil.urlEncode(Clientkey,'UTF-8'); </pre>	

In plain English: We kept the original constructor for compatibility and added a second constructor that can carry the exact Evaluation ID.

Change 2: Put the record ID into OAuth state

OLD UAT CODE	NEW UAT CODE
<pre> 16 system.debug('SF URL is: ' + sfURL); 17 </pre>	<pre> 21 system.debug('SF URL is: ' + sfURL); 22 23 + String stateValue = 'url=' + sfURL; 24 + if (String.isNotBlank(evaluationId)) { 25 + stateValue += '&evaluationId=' + evaluationId; 26 + } 27 + 28 String authuri = 'https://accounts.google.com/o/oauth2/auth?'+ </pre>

Apex Change Review

OLD UAT CODE		NEW UAT CODE	
18	String authuri = 'https://accounts.google.com/o/oauth2/auth?'+	29	'client_id='+key+
19	'client_id='+key+		

In plain English: Before leaving Salesforce for Google, the code now packs the Evaluation ID into the signed-in user's return state. Google returns that state unchanged, so Salesforce knows exactly which record to resume.

Change 3: Put the record ID into OAuth state

OLD UAT CODE		NEW UAT CODE	
21	'&scope=https://www.googleapis.com/auth/drive'+	31	'&scope=https://www.googleapis.com/auth/drive'+
22	'&redirect_uri='+uri+	32	'&redirect_uri='+uri+
23 -	'&state=security_token%3D138r5719ru3e1%26url%3D' + sfURL +	33 +	'&state=' + EncodingUtil.urlEncode(stateValue, 'UTF-8') +
24	'&login_hint=probomedical@gmail.com&' +	34	'&login_hint=probomedical@gmail.com&' +
25	'access_type=offline';	35	'access_type=offline';

In plain English: Before leaving Salesforce for Google, the code now packs the Evaluation ID into the signed-in user's return state. Google returns that state unchanged, so Salesforce knows exactly which record to resume.

5.2 cGoogleAppAuthenticationWithSalesforce

Verified change size: +153 lines / -134 lines.

Change 1: Controller safety change 1

OLD UAT CODE	NEW UAT CODE
<pre> 34 public static List<String> images { get; set; } 35 public String callfunc { get; set; } 36 - private final ProductItem__c pitem; 37 public String LensID { get; set; } 38 public String CapID { get; set; } </pre>	<pre> 34 public static List<String> images { get; set; } 35 public String callfunc { get; set; } 36 + private ProductItem__c pitem; 37 public String LensID { get; set; } 38 public String CapID { get; set; } </pre>

In plain English: This change keeps the Google callback tied to valid Salesforce data and prevents incomplete Google Drive responses from breaking the page.

Change 2: Recover and validate the Evaluation

OLD UAT CODE	NEW UAT CODE
<pre> 58 public String endoscopyRepairs { get; set; } 59 public Decimal otherEndoscopyPrice { get; set; } 60 - private final Evaluation_Natalie__c EVAL { get; set; } 61 public Decimal FullPrice { get; set; } 62 public Decimal ReqPrice { get; set; } 63 public Decimal ExcPrice { get; set; } 64 public Decimal LoanerPrice { get; set; } </pre>	<pre> 58 public String endoscopyRepairs { get; set; } 59 public Decimal otherEndoscopyPrice { get; set; } 60 + private Evaluation_Natalie__c EVAL { get; set; } 61 public Decimal FullPrice { get; set; } 62 public Decimal ReqPrice { get; set; } 63 public Decimal ExcPrice { get; set; } 64 public Decimal LoanerPrice { get; set; } 65 + 66 + @TestVisible 67 + private static String getEvaluationIdFromState(String stateValue) { 68 + if (String.isBlank(stateValue)) { 69 + return null; 70 + } 71 + 72 + String decodedState = stateValue; 73 + try { 74 + decodedState = EncodingUtil.urlDecode(decodedState, 'UTF-8'); 75 + } catch (Exception ignored) { 76 + return null; 77 + } 78 + 79 + for (String statePart : decodedState.split('&')) { 80 + List<String> nameAndValue = statePart.split('=', 2); </pre>

OLD UAT CODE	NEW UAT CODE
	<pre> 81 + if (nameAndValue.size() != 2 nameAndValue[0] != 'evaluationId') { 82 + continue; 83 + } 84 + 85 + try { 86 + Id evaluationId = Id.valueOf(nameAndValue[1]); 87 + if (evaluationId.getSObjectType() == Evaluation_Natalie__c.SObjectType) { 88 + return String.valueOf(evaluationId); 89 + } 90 + } catch (Exception ignored) { 91 + return null; 92 + } 93 + } 94 + return null; 95 + } 96 + 97 + private static void addPageError(String message) { 98 + ApexPages.addMessage(new ApexPages.Message(ApexPages.Severity.ERROR, message)); 99 + } 100 + 101 + @TestVisible 102 + private static String normalizeDriveFolderId(String folderValue) { 103 + if (String.isBlank(folderValue)) { 104 + return null; 105 + } 106 + 107 + String normalizedValue = folderValue.trim(); 108 + if (normalizedValue.contains('/folders/')) { 109 + normalizedValue = normalizedValue.substringAfter('/folders/'); 110 + } 111 + if (normalizedValue.contains('?')) { 112 + normalizedValue = normalizedValue.substringBefore('?'); 113 + } 114 + if (normalizedValue.contains('/')) { 115 + normalizedValue = normalizedValue.substringBefore('/'); 116 + } 117 + return String.isBlank(normalizedValue) ? null : normalizedValue; 118 + } 119 + 120 + private List<Product_Test_Line_Item__c> getTestItems(Id assetId) { 121 + List<ProductTest__c> recentProductTests = [SELECT Id, Product_Item__c FROM ProductTest__c WHERE Product_Item__c = :assetId ORDER BY CreatedDate DESC LIMIT 1]; </pre>

OLD UAT CODE	NEW UAT CODE
<pre> 65 private List<Product_Test_Line_Item__c> getTestItems(Id assetId) { 66 - ProductTest__c recentProductTest = [SELECT Id, Product_Item__c FROM ProductTest__c WHERE Product_Item__c = :assetId ORDER BY CreatedDate DESC LIMIT 1]; 67 system.debug('THIS IS THE TEST FOUND >> ' + recentProductTest); 68 List<Product_Test_Line_Item__c> productTestItems = [SELECT Id, Quantity__c, Item_Description__c, ProductTest__c, Complaint__c, Repair_Technician__c, Repairs_Needed__c, Repair_Price__c </pre>	<pre> 122 + if (recentProductTests.isEmpty()) { 123 + return new List<Product_Test_Line_Item__c>(); 124 + } 125 + ProductTest__c recentProductTest = recentProductTests[0]; 126 system.debug('THIS IS THE TEST FOUND >> ' + recentProductTest); 127 List<Product_Test_Line_Item__c> productTestItems = [SELECT Id, Quantity__c, Item_Description__c, ProductTest__c, Complaint__c, Repair_Technician__c, Repairs_Needed__c, Repair_Price__c </pre>

In plain English: The callback decodes the returned OAuth state, confirms the value is a real Evaluation ID, and shows a readable message instead of guessing or crashing.

Change 3: Recover and validate the Evaluation

OLD UAT CODE	NEW UAT CODE
<pre> 75 public cGoogleAppAuthenticationWithSalesforce(ApexPages.StandardController controller) { 76 itemid = ApexPages.currentPage().getParameters().get('id'); </pre>	<pre> 134 public cGoogleAppAuthenticationWithSalesforce(ApexPages.StandardController controller) { 135 itemid = ApexPages.currentPage().getParameters().get('id'); </pre>

OLD UAT CODE	NEW UAT CODE
<pre> 77 List<Product_Test_Line_Item__c> testItems = new List<Product_Test_Line_Item__c>(); 78 Id pitemid; </pre>	<pre> 136 + if (String.isBlank(itemid)) { 137 + itemid = getEvaluationIdFromState(ApexPages.currentPage().getParameters().get('state')); 138 + } 139 + 140 + FileLst = new List<String>(); 141 + FileIdAndNameMapFortheAccount = new Map<String, String>(); 142 + FileNameAndIdMapFortheAccount = new Map<String, String>(); 143 + 144 + if (String.isBlank(itemid)) { 145 + addPageError('The Evaluation could not be identified. Return to the Evaluation and start authentication again.');</pre> <pre> 146 + return; 147 + } 148 + 149 + List<Product_Test_Line_Item__c> testItems = new List<Product_Test_Line_Item__c>(); 150 + Id pitemid; </pre>

In plain English: The callback decodes the returned OAuth state, confirms the value is a real Evaluation ID, and shows a readable message instead of guessing or crashing.

Change 4: Controller safety change 4

OLD UAT CODE	NEW UAT CODE
<pre> 80 81 if (itemid != null) { 82 - EVAL = [select Id, Product_Item_ID__c, Name, Product_Family__c, Probe_Type__c, Associated_RMA__c, Quote_Issue_Date__c from Evaluation_Natalie__c WHERE Id = :itemid]; </pre> <pre> 83 84 - RMA__c associatedRMA = [</pre>	<pre> 152 153 if (itemid != null) { 154 + List<Evaluation_Natalie__c> evaluations = [select Id, Product_Item_ID__c, Name, Product_Family__c, Probe_Type__c, Associated_RMA__c, Quote_Issue_Date__c from Evaluation_Natalie__c WHERE Id = :itemid LIMIT 1]; 155 + if (evaluations.isEmpty()) { 156 + addPageError('The requested Evaluation is unavailable. Return to the Evaluation and start authentication again.');</pre> <pre> 157 + return; 158 + } 159 + EVAL = evaluations[0]; 160 161 + if (EVAL.Associated_RMA__c == null) { 162 + addPageError('The Evaluation must have an associated RMA before authentication.');</pre>

OLD UAT CODE	NEW UAT CODE
<pre> 85 select Id, Recommended_Repairs__c, Required_Repairs__c, Depot_Repair__c, Endoscopy_Evaluation_Results__c, 86 Full__c, Required__c, Exchange__c, Loaner__c, Other_Notes_Cosmetic__c, Other_Notes_Functional__c, Cause_of_Damage__c </pre>	<pre> 163 + return; 164 + } 165 + 166 + List<RMA__c> associatedRMAs = [167 select Id, Recommended_Repairs__c, Required_Repairs__c, Depot_Repair__c, Endoscopy_Evaluation_Results__c, 168 Full__c, Required__c, Exchange__c, Loaner__c, Other_Notes_Cosmetic__c, Other_Notes_Functional__c, Cause_of_Damage__c </pre>

In plain English: This change keeps the Google callback tied to valid Salesforce data and prevents incomplete Google Drive responses from breaking the page.

Change 5: Controller safety change 5

OLD UAT CODE	NEW UAT CODE
<pre> 88 where Id = :EVAL.Associated_RMA__c 89]; </pre> <pre> 90 // setting up pricing for "Other" endoscopy products 91 otherEndoscopyPrice = 0; </pre>	<pre> 170 where Id = :EVAL.Associated_RMA__c 171]; 172 + if (associatedRMAs.isEmpty()) { 173 + addPageError('The Evaluation\'s associated RMA is unavailable.');</pre> <pre> 174 + return; 175 + } 176 + RMA__c associatedRMA = associatedRMAs[0]; 177 // setting up pricing for "Other" endoscopy products 178 otherEndoscopyPrice = 0; </pre>

In plain English: This change keeps the Google callback tied to valid Salesforce data and prevents incomplete Google Drive responses from breaking the page.

Change 6: Validate the Product Item reference

OLD UAT CODE	NEW UAT CODE
<pre> 107 } 108 109 - pitemid = EVAL.Product_Item_ID__c; </pre>	<pre> 194 } 195 196 + if (String.isBlank(EVAL.Product_Item_ID__c)) { 197 + addPageError('The Evaluation must have a Product Item before authentication.');</pre>

OLD UAT CODE	NEW UAT CODE
<pre> 110 if(associatedRMA.Depot_Repair__c == false){ 111 RequiredRepairs = associatedRMA.Required_Repairs__c; </pre>	<pre> 198 + return; 199 + } 200 + try { 201 + pitemid = Id.valueOf(EVAL.Product_Item_ID__c); 202 + } catch (Exception ignored) { 203 + addPageError('The Evaluation contains an invalid Product Item reference. '); 204 + return; 205 + } 206 if(associatedRMA.Depot_Repair__c == false){ 207 RequiredRepairs = associatedRMA.Required_Repairs__c; </pre>

In plain English: The controller checks that the Evaluation has a Product Item and that its stored ID is valid before running any Google Drive logic.

Change 7: Controller safety change 7

OLD UAT CODE	NEW UAT CODE
<pre> 193 system.debug('THIS IS THE REPAIR COMMENTS >> ' + RepairComments); 194 195 - } else { // if (itemid == null) { // identifies asset by finding the most recently modified evaluation created by user 196 - 197 - Id UserID = UserInfo.getUserId(); 198 - System.debug('THIS IS THE USER >>> ' + UserID); 199 - 200 - // JMAY: rem'd out -- UserName is never used so save the SOQL 201 - //User UserName = [select id, name, profileID from User where id = :userid]; 202 - 203 - EVAL = [204 - select Id, Product_Item_ID__c, Name, Associated_RMA__c, Quote_Issue_Date__c, Product_Family__c, Probe_Type__c 205 - from Evaluation_Natalie__c 206 - WHERE LastModifiedById = :UserID 207 - ORDER BY LastModifiedDate DESC 208 - limit 1 209 -]; 210 - 211 - System.debug('THIS IS THE EVAL >>> ' + EVAL); </pre>	<pre> 289 system.debug('THIS IS THE REPAIR COMMENTS >> ' + RepairComments); 290 </pre>

OLD UAT CODE	NEW UAT CODE
<pre> 212 - pitemid = EVAL.Product_Item_ID__c; 213 - 214 - RMA__c associatedRMA = [215 - select Id, Other_Notes_Cosmetic__c, Other_Notes_Functional__c, Cause_of_Damage__c, 216 - Recommended_Repairs__c, Required_Repairs__c, Full__c, Required__c, 217 - Exchange__c, Loaner__c, Depot_Repair__c, Endoscopy_Evaluation_Results__c 218 - from RMA__c 219 - where Id = :EVAL.Associated_RMA__c 220 -]; 221 - 222 - // setting up pricing for "Other" endoscopy products 223 - otherEndoscopyPrice = 0; 224 - endoscopyRepairs = ''; 225 - if(eval.Product_Family__c != null && ((eval.Probe_Type__c != null && (!eval.Probe_Type__c.contains('Endoscopy')) && eval.Product_Family__c.contains('Endoscopy')) eval.Product_Family__c == 'Instrument Tray'))){ 226 - testItems = getTestItems(eval.Product_Item_ID__c); 227 - if(testItems.size() != 0){ 228 - for(Product_Test_Line_Item__c lineItem: testItems){ 229 - system.debug('THIS IS INSIDE FOR LOOP FOR TEST LINE ITEMS >>> ' + lineItem); 230 - if(lineItem.Repair_Price__c != null){ 231 - otherEndoscopyPrice += lineItem.Repair_Price__c; 232 - } 233 - if(lineItem.Repairs_Needed__c != null){ 234 - endoscopyRepairs += lineItem.Item_Description__c + ': ' + lineItem.Repairs_Needed__c + ' '; 235 - } 236 - } 237 - } 238 - } 239 - 240 - if(associatedRMA.Dpot_Repair__c == false){ 241 - RequiredRepairs = associatedRMA.Required_Repairs__c; 242 - RecommendedRepairs = associatedRMA.Recommended_Repairs__c; 243 - } 244 - if(associatedRMA.Dpot_Repair__c == true){ 245 - depotRequiredRepairs = associatedRMA.Required_Repairs__c; 246 - depotRecommendedRepairs = associatedRMA.Recommended_Repairs__c; 247 - } </pre>	

OLD UAT CODE	NEW UAT CODE
<pre> 248 - 249 - 250 - FullPrice = associatedRMA.Full__c; 251 - ReqPrice = associatedRMA.Required__c; 252 - ExcPrice = associatedRMA.Exchange__c; 253 - LoanerPrice = associatedRMA.Loaner__c; 254 - if(otherEndoscopyPrice != 0){ 255 - FullPrice = otherEndoscopyPrice; 256 - ReqPrice = otherEndoscopyPrice; 257 - } 258 - RepairComments = ''; 259 - EndoscopyResults = ''; 260 - EndoscopyResultsComments = ''; 261 - if(associatedRMA.Endoscopy_Evaluation_Results__c != null){ 262 - system.debug('THIS IS THE ENDOSCOPY EVAL RESULTS >> ' + associatedRMA.Endoscopy_Evaluation_Results__c + ' AND REQUIRED REPAIRS >> ' + associatedRMA.Required_Repairs__c); 263 - List<String> endoscopyResultsList = associatedRMA.Endoscopy_Evaluation_Results__c.split(';'); 264 - List<String> endoscopyRequiredRepairs = associatedRMA.Required_Repairs__c.split(';'); 265 - for(String repair:endoscopyRequiredRepairs){ 266 - String notePrefix = ''; 267 - String resultsNote = ''; 268 - if(repair.contains('Repair')){ 269 - notePrefix = repair.substringBefore('Repair').trim(); 270 - } 271 - else if(repair.contains('Replacement')){ 272 - notePrefix = repair.substringBefore('Replacement').trim(); 273 - } 274 - if(notePrefix != null && notePrefix != ''){ 275 - notePrefix = notePrefix.deleteWhitespace(); 276 - for(string result:endoscopyResultsList){ 277 - system.debug('THIS IS THE RESULT INSIDE LOOP >> ' + result + ' AND NOTE PREFIX >> ' + notePrefix); 278 - if(result.contains(notePrefix)){ 279 - resultsNote = ': ' + result.substringAfter(notePrefix).trim(); 280 - } 281 - } 282 - EndoscopyRepairsWithNotes += repair + resultsNote + '; '; 283 - } 284 - else{ 285 - EndoscopyRepairsWithNotes += repair + '; '; </pre>	

OLD UAT CODE	NEW UAT CODE
<pre> 286 - } 287 - 288 - } 289 - EndoscopyResults = associatedRMA.Endoscopy_Evaluation_Results__c; 290 - /*EndoscopyResultsComments = EndoscopyResults.replaceAll(';',' '); 291 - EndoscopyResultsComments = EndoscopyResultsComments.RemoveEnd(';'); 292 - RepairComments += 'Detailed Repair Evaluation: ' + EndoscopyResultsComments + '; '*/ 293 - } 294 - else{ 295 - if(String.isNotBlank(endoscopyRepairs)){ 296 - EndoscopyResults = endoscopyRepairs; 297 - } 298 - } 299 - 300 - if (associatedRMA.Other_Notes_Cosmetic__c != null) { 301 - RepairComments += 'Cosmetic Notes: ' + associatedRMA.Other_Notes_Cosmetic__c + '; '; 302 - } 303 - 304 - if (associatedRMA.Other_Notes_Functional__c != null) { 305 - RepairComments += 'Functional Notes: ' + associatedRMA.Other_Notes_Functional__c + '; '; 306 - } 307 - if(associatedRMA.Cause_of_Damage__c != null){ 308 - system.debug('THIS IS THE CAUSE OF DAMAGE >> ' + associatedRMA.Cause_of_Damage__c); 309 - 310 - RepairComments += 'Cause of Damage: ' + associatedRMA.Cause_of_Damage__c + '; '; 311 - } 312 - system.debug('THIS IS THE REPAIR COMMENTS >> ' + RepairComments); 313 - 314 - } 315 </pre>	

OLD UAT CODE	NEW UAT CODE
	291 } 292

In plain English: This change keeps the Google callback tied to valid Salesforce data and prevents incomplete Google Drive responses from breaking the page.

Change 8: Controller safety change 8

OLD UAT CODE	NEW UAT CODE
<pre>317 System.debug('THIS IS THE PITEMID >>>> ' + pitemid); 318 319 - pitem = [select Id, Google_Drive_Folder_ID__c, Product_Family__c from ProductItem__c where Id = :pitemid]; 320 321 322 - DriveFolder = pitem.Google_Drive_Folder_ID__c; 323 System.debug('THIS IS THE DRIVE FOLDER >>>> ' + DriveFolder);</pre>	<pre>294 System.debug('THIS IS THE PITEMID >>>> ' + pitemid); 295 296 + List<ProductItem__c> productItems = [select Id, Google_Drive_Folder_ID__c, Product_Family__c from ProductItem__c where Id = :pitemid LIMIT 1]; 297 + if (productItems.isEmpty()) { 298 + addPageError('The Evaluation\'s Product Item is unavailable.');</pre> <pre>299 + return; 300 + } 301 + pitem = productItems[0]; 302 303 304 + DriveFolder = normalizeDriveFolderId(pitem.Google_Drive_Folder_ID__c); 305 System.debug('THIS IS THE DRIVE FOLDER >>>> ' + DriveFolder);</pre>

OLD UAT CODE	NEW UAT CODE
324	306

In plain English: This change keeps the Google callback tied to valid Salesforce data and prevents incomplete Google Drive responses from breaking the page.

Change 9: Stop an invalid save safely

OLD UAT CODE	NEW UAT CODE
<pre> 341 342 public PageReference save() { 343 344 EVAL.Lens_ID__c = LensID; </pre>	<pre> 323 324 public PageReference save() { 325 + 326 + if (EVAL == null) { 327 + addPageError('No Evaluation is available to save. Return to the Evaluation and try again.');</pre> <pre> 328 + return null; 329 + } 330 331 EVAL.Lens_ID__c = LensID; </pre>

In plain English: If the callback did not load a valid Evaluation, Save now stops with a clear message instead of dereferencing a null record.

Change 10: Send the exact Evaluation into Google authentication

OLD UAT CODE	NEW UAT CODE
<pre> 382 public PageReference DriveAuth() { 383 384 - PageReference pg = new PageReference(new cAuthURIForEval(key, redirect_uri).AuthenticationURI) ; 385 386 return pg; </pre>	<pre> 369 public PageReference DriveAuth() { 370 371 + if (String.isBlank(itemid)) { 372 + addPageError('The Evaluation could not be identified. Return to the Evaluation and try again.');</pre> <pre> 373 + return null; 374 + } 375 + 376 + PageReference pg = new PageReference(new cAuthURIForEval(key, redirect_uri, itemid).AuthenticationURI) ; 377 378 return pg; </pre>

In plain English: The Authenticate action now passes the current Evaluation ID into the OAuth URL builder. This is the hand-off that keeps the record identity intact during the round trip.

Change 11: Require a usable folder ID

OLD UAT CODE	NEW UAT CODE
<pre> 421 422 public PageReference ListFiles() { 423 424</pre>	<pre> 413 414 public PageReference ListFiles() { 415 + 416 + if (String.isBlank(DriveFolder)) { 417 + addPageError('The Product Item does not have a valid Google Drive folder.');</pre> <pre> 418 + return null; 419 + } 420 421</pre>

In plain English: The file lookup now stops early and explains the problem when the Product Item has no valid Drive folder.

Change 12: Ignore empty Drive file names

OLD UAT CODE	NEW UAT CODE
<pre> 465 System.debug('THIS IS THE FILE ID MAP >>>> ' + FileIdAndNameMap); 466 for (String s : FileLst) { 467 - FileIdAndNameMapFortheAccount.put(s, FileIdAndNameMap.get(s)); 468 - FileNameAndIdMapFortheAccount.put(FileIdAndNameMap.get(s), s); 469 - folderdate.add(FileIdAndNameMap.get(s)); 470 } 471 if(Test.isRunningTest()){</pre>	<pre> 462 System.debug('THIS IS THE FILE ID MAP >>>> ' + FileIdAndNameMap); 463 for (String s : FileLst) { 464 + String mappedFileName = FileIdAndNameMap.get(s); 465 + if (String.isBlank(mappedFileName)) { 466 + continue; 467 + } 468 + FileIdAndNameMapFortheAccount.put(s, mappedFileName); 469 + FileNameAndIdMapFortheAccount.put(mappedFileName, s); 470 + folderdate.add(mappedFileName); 471 } 472 if(Test.isRunningTest()){</pre>

In plain English: Google responses can contain missing or incomplete items. The controller now skips blank names instead of adding bad map entries.

Change 13: Protect dated-folder parsing

OLD UAT CODE		NEW UAT CODE	
491	String correctdate = '';	492	String correctdate = '';
492	for (String f : folderdate) {	493	for (String f : folderdate) {
		494 +	if (String.isBlank(f) f.length() < 6) {
		495 +	continue;
		496 +	}
		497	folderdateshort = f.left(6);
493	folderdateshort = f.left(6);	498	if (folderdateshort.isNumeric()) {
494	if (folderdateshort.isNumeric()) {		

In plain English: The old code always took the first six characters. The new guard skips short or blank names and avoids substring errors.

Change 14: Protect dated-folder parsing

OLD UAT CODE		NEW UAT CODE	
500	}	504	}
501	for (String f : folderdate) {	505	for (String f : folderdate) {
		506 +	if (String.isBlank(f) f.length() < 6) {
		507 +	continue;
		508 +	}
		509	if (f.left(6) == String.valueOf(baseline)) {
502	if (f.left(6) == String.valueOf(baseline)) {	510	correctdate = f;
503	correctdate = f;		

In plain English: The old code always took the first six characters. The new guard skips short or blank names and avoids substring errors.

Change 15: Explain when no image folder matches

OLD UAT CODE		NEW UAT CODE	
505	}	512	}
506	DriveFolder = FileNameAndIdMapFortheAccount.get(correctdate);	513	DriveFolder = FileNameAndIdMapFortheAccount.get(correctdate);
		514 +	if (String.isBlank(DriveFolder)) {
		515 +	addPageError('No valid image folder was found in Google
		Drive.');	
		516 +	return null;
		517 +	}

OLD UAT CODE		NEW UAT CODE	
507	System.debug('THIS IS THE DRIVE FOLDER ID >>>> ' +	518	System.debug('THIS IS THE DRIVE FOLDER ID >>>> ' +
DriveFolder);		DriveFolder);	
508	HttpRequest req3 = new HttpRequest();	519	HttpRequest req3 = new HttpRequest();

In plain English: If no valid dated image folder can be selected, the user receives a clear message and the callout is not attempted with a blank folder ID.

Change 16: Clean nested-folder file results

OLD UAT CODE		NEW UAT CODE	
544	for (String s : FileLst) {	555	for (String s : FileLst) {
545	System.debug('Checking S for Map: \'' + s + '\');	556	System.debug('Checking S for Map: \'' + s + '\');
546 -	System.debug('Adding this Product to Map2: \'' +	557 +	String nestedMappedFileName = FileIdAndNameMap2.get(s);
FileIdAndNameMap2.get(s).trim() + '\');		558 +	if (String.isBlank(nestedMappedFileName)) {
547 -	FileIdAndNameMapFortheAccount.put(s,	559 +	continue;
FileIdAndNameMap2.get(s).trim());		560 +	}
548 -	FileNameAndIdMapFortheAccount.put(FileIdAndNameMap2.get(s).trim(), s);	561 +	nestedMappedFileName = nestedMappedFileName.trim();
		562 +	System.debug('Adding this Product to Map2: \'' +
		nestedMappedFileName + '\');	
		563 +	FileIdAndNameMapFortheAccount.put(s,
		nestedMappedFileName);	
		564 +	FileNameAndIdMapFortheAccount.put(nestedMappedFileName,
		s);	
		565	}
		566	}
549	}		
550	}		

In plain English: The same blank-name protection is applied to files found inside the selected image folder.

Change 17: Make file mocks deterministic

OLD UAT CODE		NEW UAT CODE	
657		673	
658	}	674	}
		675 +	if (Test.isRunningTest() && testFileMap != null) {
		676 +	return testFileMap.clone();
		677 +	}
		678	return FilePropertiesDetails;
659	return FilePropertiesDetails;	679	}

Apex Change Review

OLD UAT CODE	NEW UAT CODE
660 }	

In plain English: During tests, the controller returns the prepared file map directly. That lets the test exercise filename matching without making a real Google call.

5.3 GoogleAuthTestClass

Verified change size: +92 lines / -7 lines.

Change 1: Add a safe test folder URL

OLD UAT CODE	NEW UAT CODE
<pre> 70 insert pro; 71 72 - ProductItem__c prItem = new ProductItem__c (Product_Name__c=pro.Id,Model__c='Philips C5-1', 73 - Serial_No__c='B0K81J',Functional_Rating__c='B',Functional_Notes__c='Good', Check_In_Type__c = 'Customer Repair', 74 - Cosmetic_Rating__c='C',Cosmetic_Notes__c='Yellow Cable',/*Location__c='WIP',*/PO_Number__c='2425', 75 - Other_Notes__c='Cable needs cleaned',Added_To_Opportunity__c = false,Receive_date__c=Date.parse('01/08/2015')); 76 insert prItem; 77 </pre>	<pre> 70 insert pro; 71 72 + ProductItem__c prItem = new ProductItem__c (Product_Name__c=pro.Id,Model__c='Philips C5-1', 73 + Serial_No__c='B0K81J',Functional_Rating__c='B',Functional_Notes__c='Good', Check_In_Type__c = 'Customer Repair', 74 + Cosmetic_Rating__c='C',Cosmetic_Notes__c='Yellow Cable',/*Location__c='WIP',*/PO_Number__c='2425', 75 + Other_Notes__c='Cable needs cleaned',Added_To_Opportunity__c = false,Receive_date__c=Date.parse('01/08/2015'), 76 + Image__c='https://example.invalid/folders/test- folder?resourcekey=test'); 77 insert prItem; 78 </pre>

In plain English: The unit test now includes a fake folder URL with a query string so folder-ID cleanup is tested without using a real Google Drive folder.

Change 2: Prove only the intended Evaluation changes

OLD UAT CODE	NEW UAT CODE
<pre> 104 Evaluation_Natalie__c e = new Evaluation_Natalie__c (Associated_RMA__c=newRMA.id); 105 insert e; 106 - 107 - //apexPages.currentPage().getParameters().put('id',e.Id); </pre>	<pre> 105 Evaluation_Natalie__c e = new Evaluation_Natalie__c (Associated_RMA__c=newRMA.id); 106 insert e; 107 + 108 + // A second record proves the OAuth callback does not use the former 109 + // "latest Evaluation modified by this user" fallback. 110 + Evaluation_Natalie__c competingEvaluation = new Evaluation_Natalie__c(Associated_RMA__c=newRMA.id); 111 + insert competingEvaluation; 112 + </pre>

OLD UAT CODE		NEW UAT CODE
108	test.setMock(HttpCalloutMock.class, new RestMock());	113 + String callbackState = 'url=https://example.invalid/apex/AuthenticationGoogleDrive&evaluationId=' + e.Id;
109	Test.StartTest();	114 + ApexPages.currentPage().getParameters().put('state', EncodingUtil.urlEncode(callbackState, 'UTF-8')); 115 test.setMock(HttpCalloutMock.class, new RestMock()); 116 Test.StartTest();

In plain English: The test creates a target Evaluation and a second control Evaluation. It rebuilds the callback state, runs the controller, and verifies the target changes while the control record remains untouched.

Change 3: Test coverage change 3

OLD UAT CODE		NEW UAT CODE
111	ApexPages.StandardController sc = new ApexPages.StandardController(e);	118 ApexPages.StandardController sc = new ApexPages.StandardController(e);
112	cGoogleAppAuthenticationWithSalesforce testGoogle = new cGoogleAppAuthenticationWithSalesforce(sc);	119 cGoogleAppAuthenticationWithSalesforce testGoogle = new cGoogleAppAuthenticationWithSalesforce(sc);
113	testGoogle.testFileMap = new Map<String, String>{'C-Lens.JPG' => 'test10', 'C-Cap.JPG' => 'test11', 'C-Whole.JPG' => 'test12',	120 + System.assertEquals(String.valueOf(e.Id), cGoogleAppAuthenticationWithSalesforce.itemid, 121 + 'The callback must restore the exact Evaluation from OAuth state.');
114	'C-SN.JPG' => 'test13', 'C-Damage 1.JPG' => 'test14', 'C- Damage 2.JPG' => 'test15', 'C-Damage 3.JPG' => 'test16',	122 + System.assertEquals('0BzmqJILtsGQ8NTZ3ZXBKYN4U3c', 123 + cGoogleAppAuthenticationWithSalesforce.normalizeDriveFolderId('0BzmqJILtsGQ8NTZ3 ZXBKYN4U3c?resourcekey=test'), 124 + 'The resource-key query string must not become part of the folder ID.');
115	'F-Leakage Test.JPG' => 'test17', 'F-Phantom 1.JPG' => 'test18', 'F-Airscan.JPG' => 'test19', 'F-Phantom 2.JPG' => 'test20',	125 testGoogle.testFileMap = new Map<String, String>{'C-Lens.JPG' => 'test10', 'C-Cap.JPG' => 'test11', 'C-Whole.JPG' => 'test12', 126 'C-SN.JPG' => 'test13', 'C-Damage 1.JPG' => 'test14', 'C- Damage 2.JPG' => 'test15', 'C-Damage 3.JPG' => 'test16', 127 'F-Leakage Test.JPG' => 'test17', 'F-Phantom 1.JPG' => 'test18', 'F-Airscan.JPG' => 'test19', 'F-Phantom 2.JPG' => 'test20',
116 -	'First Call.JPG' => 'test21', 'test10' => 'newtest', 'test11' => 'newtest2', 'test12' => 'newtest3'};	128 + 'First Call.JPG' => 'test21', 'test10' => 'C-Lens.JPG', 'test11' => 'C-Cap.JPG', 'test12' => 'C-Whole.JPG';
117	testGoogle.testFileList = new List<String>{'test10', 'test11', 'test12'};	129 testGoogle.testFileList = new List<String>{'test10', 'test11', 'test12'};
118	testGoogle.DriveAuth();	130 testGoogle.DriveAuth();

In plain English: This test change checks the new callback and folder-handling behavior without relying on a live Google account.

Change 4: Prove only the intended Evaluation changes

OLD UAT CODE		NEW UAT CODE	
120	testGoogle.AccessToken();	132	testGoogle.AccessToken();
121	testGoogle.ListFiles();	133	testGoogle.ListFiles();
		134 +	
		135 +	// Exercise the valid dated-subfolder path independently of the direct-image path.
		136 +	testGoogle.DriveFolder = 'root-folder';
		137 +	testGoogle.FileIdAndNameMapForTheAccount = new Map<String, String>();
		138 +	testGoogle.FileNameAndIdMapForTheAccount = new Map<String, String>();
		139 +	testGoogle.testFileList = new List<String>{'folder-key'};
		140 +	testGoogle.testFileMap = new Map<String, String>{
		141 +	'folder-key' => '240101 Images',
		142 +	'240101 Images' => 'folder-key'
		143 +	};
		144 +	testGoogle.ListFiles();
		145 +	testGoogle.LensID = 'case-00010716';
		146 +	testGoogle.CapID = 'cap-image';
		147 +	testGoogle.Damage1 = 'case-00010716';
		148	testGoogle.save();
		149	
		150	Test.StopTest();
122	testGoogle.save();	151 +	
123		152 +	Map<Id, Evaluation_Natalie__c> savedEvaluations = new Map<Id, Evaluation_Natalie__c>([
124	Test.StopTest();	153 +	SELECT Id, Lens_ID__c
		154 +	FROM Evaluation_Natalie__c
		155 +	WHERE Id IN :new Set<Id>{e.Id, competingEvaluation.Id}
		156 +	]);
		157 +	System.assertEquals('case-00010716',
			savedEvaluations.get(e.Id).Lens_ID__c,
		158 +	'The intended Evaluation must be updated after the OAuth callback.');
		159 +	System.assertEquals(null,
			savedEvaluations.get(competingEvaluation.Id).Lens_ID__c,
		160 +	'A newer Evaluation must not be selected by accident.');
		161	});
		162	static testMethod void mytest3(){
125	}		
126	static testMethod void mytest3(){		

In plain English: The test creates a target Evaluation and a second control Evaluation. It rebuilds the callback state, runs the controller, and verifies the target changes while the control record remains untouched.

Change 5: Test missing and invalid inputs

OLD UAT CODE	NEW UAT CODE
<pre> 187 test.save(); 188 } </pre>	<pre> 223 test.save(); 224 } 225 + 226 + @isTest 227 + static void case00010716RejectsMissingOrInvalidCallbackState() { 228 + Test.setCurrentPage(new PageReference('/apex/AuthenticationGoogleDrive')); 229 + 230 + ApexPages.StandardController standardController = 231 + new ApexPages.StandardController(new Evaluation_Natalie__c()); 232 + cGoogleAppAuthenticationWithSalesforce controller = 233 + new cGoogleAppAuthenticationWithSalesforce(standardController); 234 + 235 + System.assertEquals(null, controller.DriveAuth(), 236 + 'Authentication must not start without a specific Evaluation.');</pre> <pre> 237 + System.assertEquals(null, controller.save(), 238 + 'A callback without an Evaluation must fail safely rather than dereference null.');</pre> <pre> 239 + controller.DriveFolder = null; 240 + System.assertEquals(null, controller.ListFiles(), 241 + 'A missing Drive folder must fail safely rather than make a callout.');</pre> <pre> 242 + System.assert(ApexPages.getMessages().size() > 0, 243 + 'The page must provide a user-readable error.');</pre> <pre> 244 + 245 + System.assertEquals(null, 246 + cGoogleAppAuthenticationWithSalesforce.getEvaluationIdFromState(null)); 247 + System.assertEquals(null, 248 + cGoogleAppAuthenticationWithSalesforce.getEvaluationIdFromState('evaluationId=no t-an-id')); 249 + System.assertEquals(null, 250 + cGoogleAppAuthenticationWithSalesforce.getEvaluationIdFromState('%ZZ')); 251 + System.assertEquals(null, </pre>

OLD UAT CODE	NEW UAT CODE
189 190	<pre> 252 + cGoogleAppAuthenticationWithSalesforce.getEvaluationIdFromState(253 + 'evaluationId=' + UserInfo.getUserId()); 254 + System.assertEquals(null, 255 + cGoogleAppAuthenticationWithSalesforce.getEvaluationIdFromState('other=value')); 256 + System.assertEquals(null, 257 + cGoogleAppAuthenticationWithSalesforce.normalizeDriveFolderId(null)); 258 + System.assertEquals('folder-id', 259 + cGoogleAppAuthenticationWithSalesforce.normalizeDriveFolderId(260 + 'https://drive.google.com/drive/folders/folder- id?resourcekey=test')); 261 + System.assertEquals('folder-id', 262 + cGoogleAppAuthenticationWithSalesforce.normalizeDriveFolderId('folder- id/child')); 263 + cGoogleAppAuthenticationWithSalesforce.required = new List<String>{'required-test'}; 264 + cGoogleAppAuthenticationWithSalesforce.recommended = new List<String>{'recommended-test'}; 265 + cGoogleAppAuthenticationWithSalesforce.images = new List<String>{'image-test'}; 266 + controller.callfunc = 'callback-test'; 267 + controller.eitemid = 'evaluation-test'; 268 + System.assertEquals('required-test', cGoogleAppAuthenticationWithSalesforce.required[0]); 269 + System.assertEquals('recommended-test', cGoogleAppAuthenticationWithSalesforce.recommended[0]); 270 + System.assertEquals('image-test', cGoogleAppAuthenticationWithSalesforce.images[0]); 271 + System.assertEquals('callback-test', controller.callfunc); 272 + System.assertEquals('evaluation-test', controller.eitemid); 273 + } 274 275 </pre>

In plain English: The new negative test confirms that missing IDs, invalid callback state, wrong object IDs, and missing folders produce safe results and readable messages.

6. Verification

7. Old side: retrieved from ProboUAT before deployment.
8. New side: retrieved again from ProboUAT after deployment.
9. Deployment 0AfjH0000000T6rSAE succeeded with 3/3 components.
10. GoogleAuthTestClass completed 4/4 tests with no failures.
11. All changed Apex hunks are included in the side-by-side section.
12. No unchanged OAuth credential lines are reproduced in this document.

7. Bottom line

Before the fix, the Google callback could return without knowing which Evaluation started the process and could choose the wrong recent record. After the fix, the exact Evaluation travels through OAuth, is validated on return, and is the only record the save test allows to change. Folder links and incomplete Drive results are also handled more safely.